CPPCON 2026 · DAMIEN BUHL

Automatic Splitting of Translation Units

Compiler-Agnostic Compiler Frontend Parallelization

CO-FOUNDER TIPI.BUILD BY ENGFLOW

Damien Buhl
@daminetreg

EngFlow

What I'm passionate about:

Boring and Slow builds?

No Space Left on Device? AI Agent producing too much code to build?

CI Cloud Costs too high ?

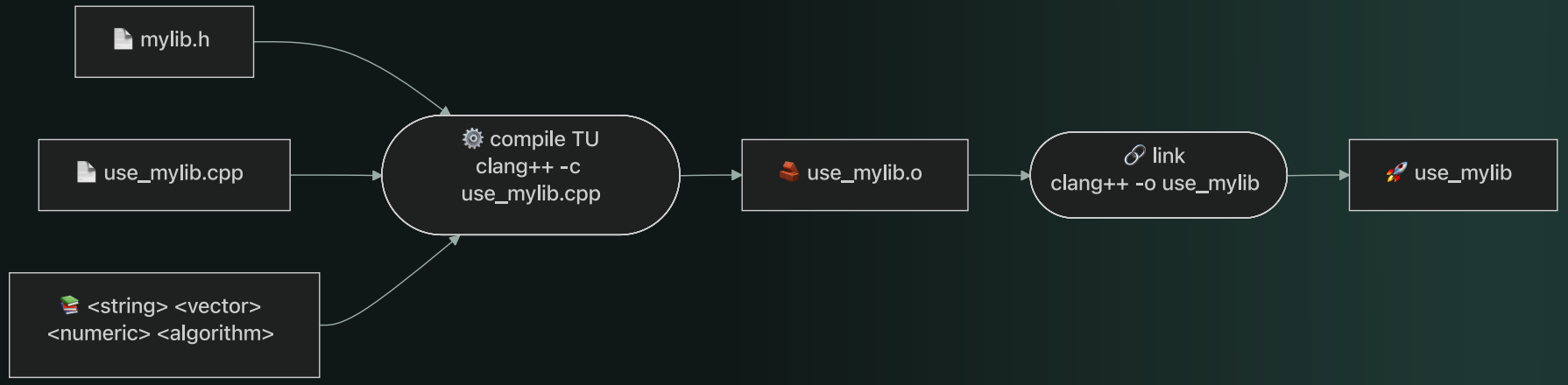
On-prem CPUs waiting in queue?

Modernizing / Migrating CMake or Bazel?

C++ and Pragmatic Safety with Sanitizers builds

TRANSLATION UNIT (TU) FISSION

A build is a graph.



The build system reruns when a file changes; The atoms of builds are TUs (Translation Units).

It's the finest-granular action that can be taken in a C++ build.

TRANSLATION UNIT (TU) FISSION

TUs sizing

- Why are TUs the size they are ?
  - intra-TU compiler optmization (Before LTO)
  - all function of class/module belong logically together
  - the developer (or the LLM) was just too lazy to create another file

LTO: Link Time Optimization

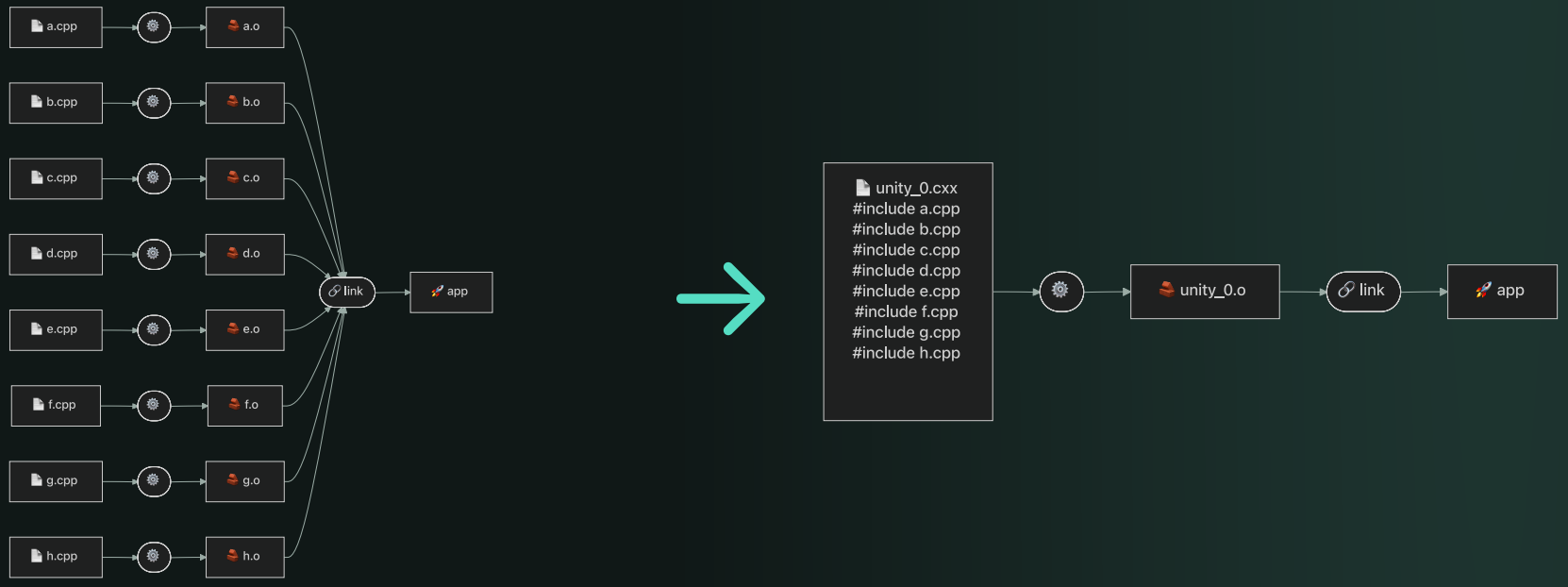
TRANSLATION UNIT (TU) FISSION

Build graph optimization techniques.

- **Unity builds** — TUs fusion: fewer, bigger TUs.
  - concatenating TUs
- **Splitting / Partitioning / Sharding** — TUs fission: more, smaller TUs.
  - splitting TUs per function/function groups
  - taking bodies out of headers
  - splitting headers : forward declaration to limit inclusions (c.f. Fast Kernel Headers Tree v1 - Ingo Molnar 2021)

TRANSLATION UNIT (TU) FISSION

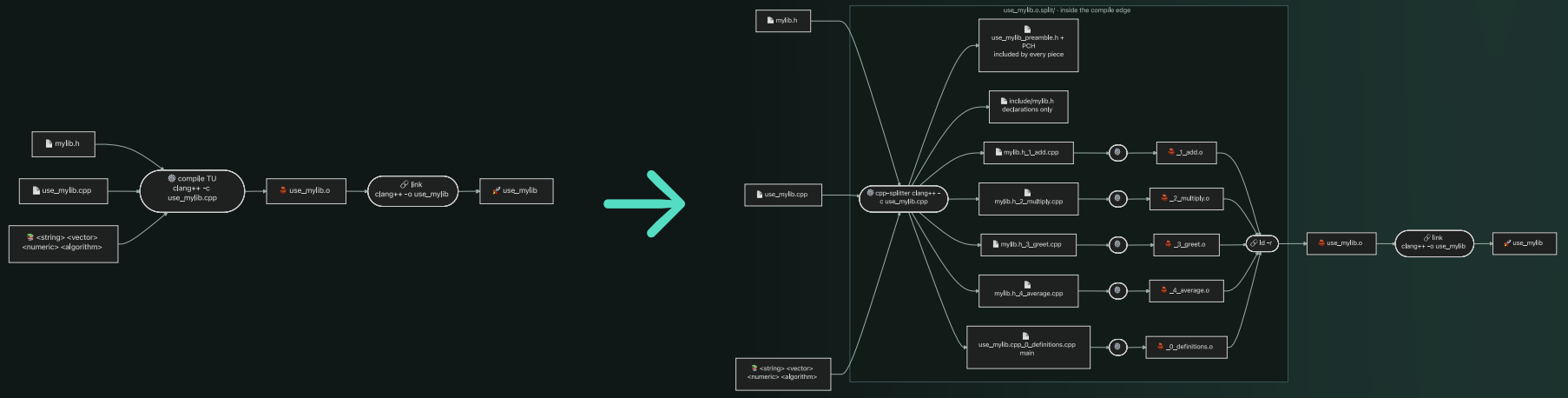
TUs fusion : Unity Builds



- Use Cases
  - Before LTO: Permitted more intra-TU optimization
  - Faster builds on limited CPU cores count: amortizes preprocessing

TRANSLATION UNIT (TU) FISSION

TUs fission : Splitting TUs, Headers & Modules



- Use Cases
  - Faster builds on parallel distributed CPU cores
  - Faster edit-build-test loop

WETHER-TO

To unity- or split- build ?

- **It depends** :
  - Compiling for an old platform without LTO ?
  - Build farm available, or just a 12-cores laptop ?
- **Unity builds**
  - **Requires** TUs to be concatenable:
    - *static* types and functions, *anonymous namespace* : **collisions**.
- **Split builds**
  - Copy/paste manually definitins from headers, adapt declaration, cut translation units in smaller ones.
    - Or Ask your AI Agent to burn tokens?
    - Will the new file layout make any sense architecturally / logically anymore ?
    - What if an Unity Build later is useful ?

HOW-TO

How to unity- or split- ?

- **Unity builds**
  - `cmake ... -DCMAKE_UNITY_BUILD=ON`
  - Back in 2018 we built a deterministic tool to automatically handle *static* types and functions, *anonymous namespace* collisions
    - tipi.build binary optimizer / libclang based "headerizer"
- **Split builds**
  - `cmake ... -DCMAKE_CXX_COMPILER_LAUNCHER=cpp-splitter`
  - `github.com/daminetreg/splitter`

> 🥁 Compiler-Agnostic Compiler Frontend Parallelization

github.com/daminetreg/splitter

⭐ ✨ Github stars welcome, alongside issues and PRs !💫

Example Walkthrough :

From a monolithic header-only and single TU build to a splitted build graph...

TRANSLATION UNIT (TU) FISSION

mylib.h an header only library

```
// mylib.h
#pragma once
#include <string>
#include <vector>
#include <numeric>
#include <algorithm>

inline int add(int a, int b) {
    return a + b;
}

inline int multiply(int a, int b) {
    return a * b;
}
```

```
inline std::string greet(const std::string&
name) {
    return "Hello, " + name + "!";
}

inline double average(const
std::vector<double>& v) {
    if (v.empty()) return 0.0;
    double sum = std::accumulate(
        v.begin(), v.end(), 0.0);
    return sum / v.size();
}

template<typename T>
T max_of(T a, T b) {
    return (a > b) ? a : b;
}
```

TRANSLATION UNIT (TU) FISSION

use_mylib.cpp a monolihic TU

```cpp
#include "mylib.h"

int main() {
  add(3, 4);
  multiply(5, 6);
  greet("World");

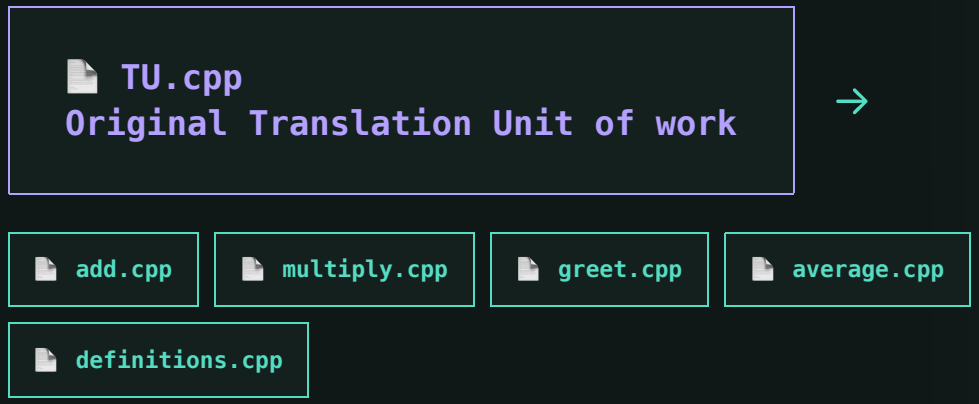
  std::vector<double> vals = {1.0, 2.0, 3.0, 4.0, 5.0};
  average(vals);

  max_of(10, 20);

    return 0;
}
```

TRANSLATION UNIT (TU) FISSION

Deterministic: Splitting TUs, Headers & Modules



Increasing atomicity of builds. Split the atom: one piece per function.

Goal: Finer-grained compilation. Finer-grained caching. Finer-grained distribution.

CMAKE_CXX_COMPILER_LAUNCHER

Integrates transparently

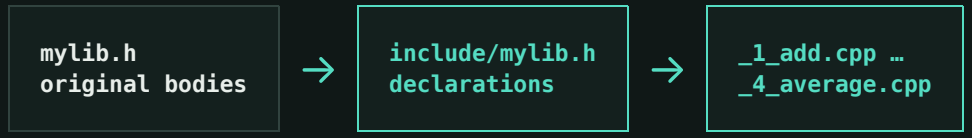
`-DCMAKE_CXX_COMPILER_LAUNCHER=cpp-splitter`, will return a relocatable use_mylib.o.

```
cpp-splitter clang++ -I. -MD -MF use_mylib.o.d -c -o use_mylib.o use_mylib.cpp
```

```
use_mylib.o.split/
├── prefix.h · libclang PCH
├── preamble.h · compiler PCH
├── definitions header + owner
├── one source + object per piece, under the mirrored include/ tree
│   ├── mylib.h · rewritten: declarations only, stripped definitions
│   ├── mylib.h_1_add.cpp → .o
│   ├── mylib.h_2_multiply.cpp → .o
│   ├── mylib.h_3_greet.cpp → .o
│   ├── mylib.h_4_average.cpp → .o
│   └── max_of · template, kept in mylib.h, no piece
└── cache · hash · harvest · depfile
```

HEADER SHADOWING + MIRRORING

Header shadowing + splitting



The split include root precedes project include directories.

CLASSIFICATION

Not every body is splittable

splitting TUs = moving + keeping functions

moved

Ordinary emitted definitions become pieces.

kept

main entrypoint · templates · internal header linkage · macro-bound extents · non emitted inline functions

> Keep forms whose semantics depend on instantiation, linkage, class syntax, or unreplayable macro context.

TWO PREAMBLES

PCH preamble

Parse common includes and declarations **once** across split-body pieces

use_mylib_preamble.h

Common Includes + declarations. Every split-body includes it; its matching compiler PCH is reused.

use_mylib.cpp_definitions.h

Kept external definitions, including main. Included by exactly one owner piece.

> Share parsed declarations through a compiler PCH while assigning unique definitions that may exist once to a single, explicit object owner.

`PCH = Pre Compiled Header`

HEADER PIECE

split-body pieces

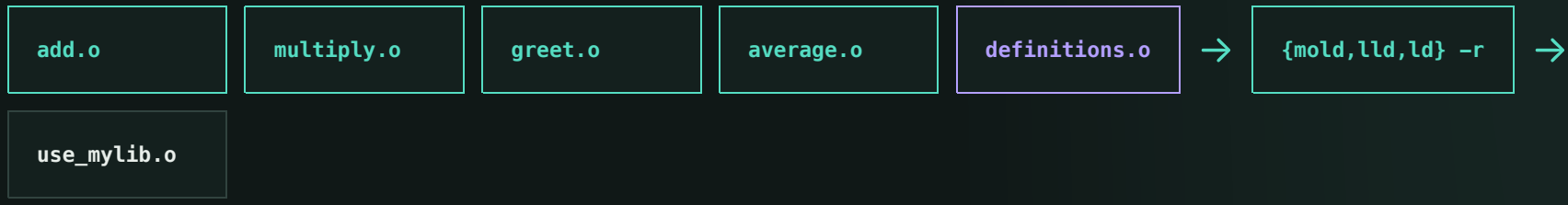
```
// include/mylib.h_1_add.cpp
#include "use_mylib_preamble.h"
#include "mylib.h"

__attribute__((used))
#line 7 "/…/mylib.h"
inline int add(int a, int b) { return a + b; }
```

> Mirror included paths and compile each moved body under its consumer's declaration context so resolution remains equivalent.

RECOMPOSE

5 objects builds in parallel
1 ordinary object out.



> Use relocatable linking to return precisely the conventional object file expected by every downstream build step.

modules are the same

```
// foo.cxx — the interface unit
module;
#include <iostream>

export module foo;

export class foo {
public:
  foo();
  ~foo();
  void helloworld();
};
```

```
// …and its bodies, in the same file
foo::foo() = default;
foo::~foo() = default;
void foo::helloworld() {
  std::cout << "hello world\n";
}

// every body edit changes the BMI:
// its ODR hash is stored, even with
// -fmodules-reduced-bmi
```

> No header reparsed by importers — but the BMI they read changes on every body edit, non-inline bodies included. So let's autosplit them before we, or the LLM, adopt them: we or the agent will write everything in the same file, all lazy.

MODULES

Integrates transparently with CMake modules support

```
cpp-splitter clang++ -std=c++20 @foo.cxx.o.modmap -c -o foo.cxx.o foo.cxx
// foo.cxx.o.modmap:  -x c++-module -fmodule-output=foo.pcm -fmodules-reduced-bmi
```

```
foo.cxx.o.split/
├── foo_interface.cxx · declarations only → foo.pcm + foo_interface.o
├── foo_preamble.h · the global module fragment, replayed by every piece
├── foo.cxx_1_foo.cpp · foo.cxx_2_~foo.cpp · foo.cxx_3_helloworld.cpp
│ · one implementation unit per body, compiled with -fmodule-file=foo=foo.pcm
├── cache · hash · harvest · keeps · depfile (names foo.cxx and foo.pcm)
└── *.o · ld -r → foo.cxx.o
```

MODULES · INTERFACE

Modules shadowing + splitting



The compiler precompiles the rewritten interface and shadow it over the original module; a body edit leaves foo.pcm byte-identical.

MODULES · PIECE

Interface only BMI

Shared global module fragment with preamble.

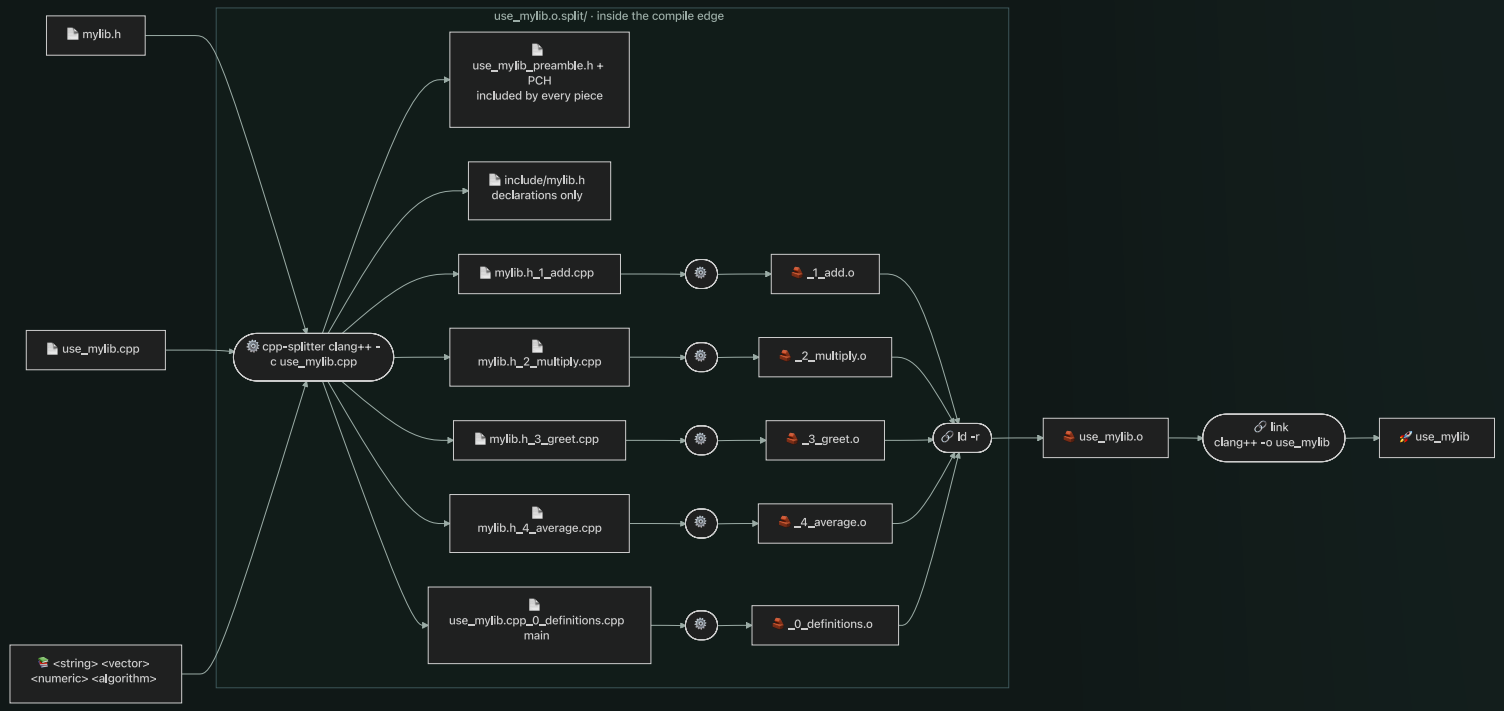
```
// foo.cxx.o.split/foo.cxx_3_helloworld.cpp
module;
#include "foo_preamble.h"     // the global module fragment: <iostream>
module foo;                   // implementation unit: imports foo.pcm implicitly

#line 17 "/…/foo.cxx"
void foo::helloworld() { std::cout << "hello world\n"; }
```

> Each moved body compiled as an implementation unit under its own interface, with the global module fragment replayed: the BMI never carries the body.

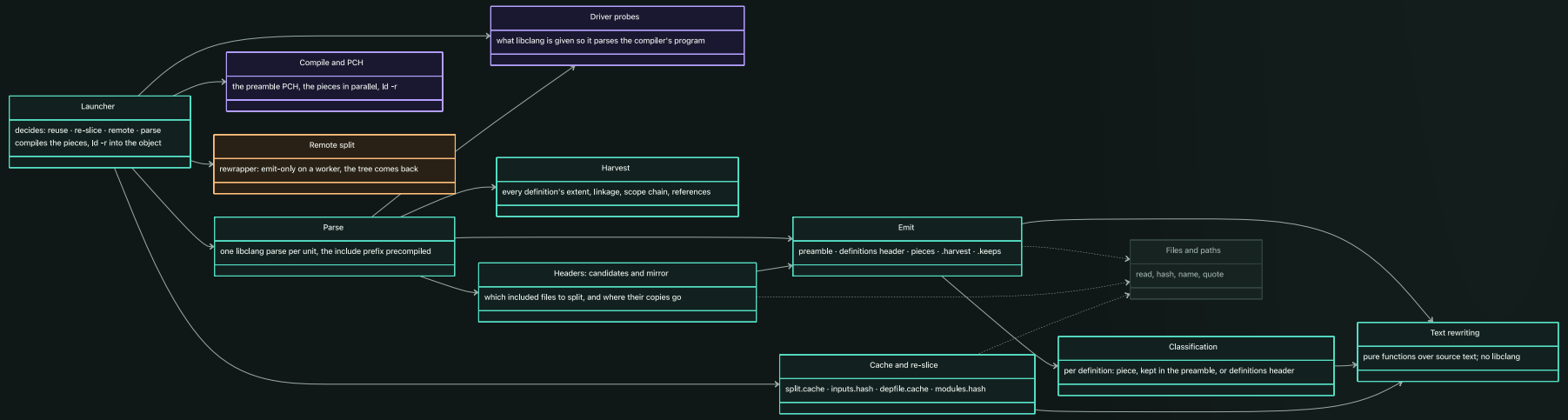
SPLIT GRAPH

Same build, parallel graph.



Same inputs, same object out. Original TU compilation, expands to one compile per body.

Architecture & Demo



PARSE AND RE-SLICING

Harvest function body locations and emission.

include prefix → libclang PCH hash + deps → one AST parse →

definitions USR + bytes extent/location
+ whether inline body are called and hence should/should not be emitted

> USR = Unified Symbold Resolution, Clang AST unique entity identifier (*e.g.* `c:@N@demo@S@Widget@F@value#1` for `demo::Widget::value() const`)

PARSE AND RE-SLICING

Avoid reparsing to change only a body

depfile.cache

Original prerequisites remain visible to the build.

inputs.hash

Content hashes for prerequisites.

harvest

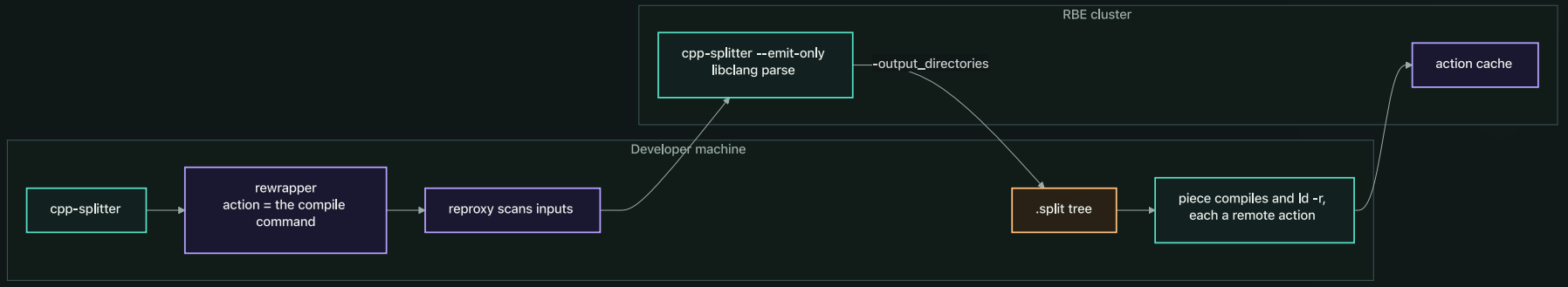
Function body locations and their body/signature hashes

> Record content and source geometry (extents) so a body-only change can recreate the same split tree without rebuilding the AST: re-hash + split + rebuild only the changed body.

REMOTE EXECUTION & CACHING

Remote C++ splitting

Leverage CMake RE's rewrapper to distribute and cache splitting.



> Parsing C++ requires alot of RAM depending the codebase, leverage scalable build distribution cluster with Bazel RBE when run with CMake RE, also with remote compilation and scheduling.

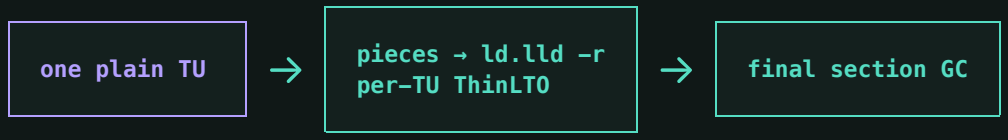
Impact on the resulting Binary

Optimized builds possible ? Restrict it to developer edit loop only ?

BINARY EQUALITY

Comparing split to plain

```
plain -O2 -ffunction-sections -fdata-sections -Wl,--gc-sections
split same flags + -flto=thin · CPP_SPLITTER_LINKER=ld.lld
```

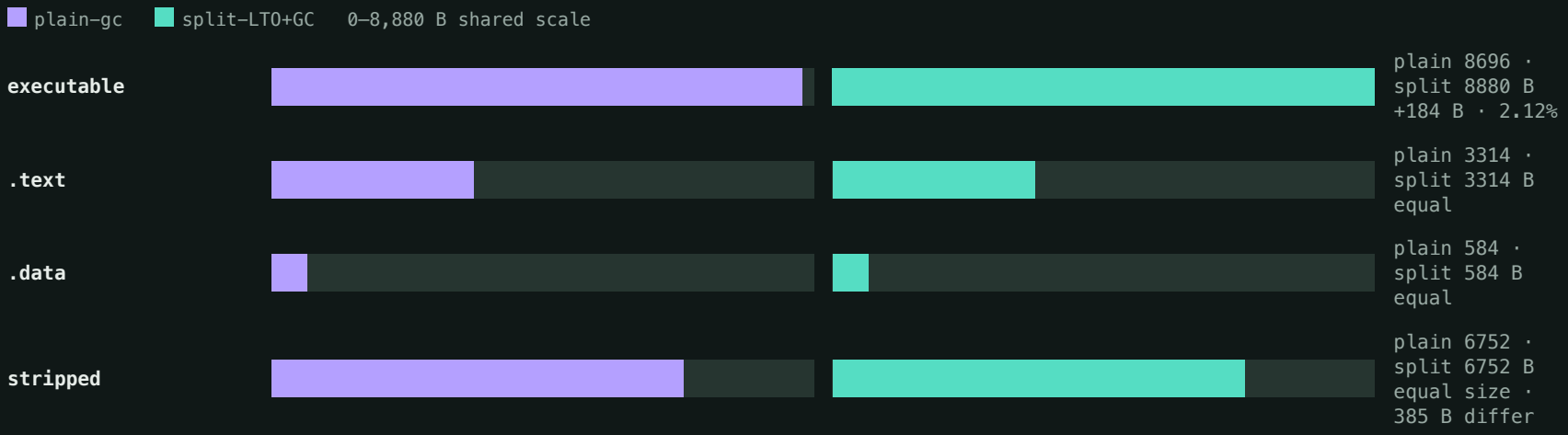


The baseline does not use LTO. lld relinks the split unit to a native object; final linking does no LTO across units.

BINARY EQUALITY

Same binary different metadata.

all metrics | executable | .text | stripped



22 symbols vs 22; normalized instructions 178 vs 178. Stripped: 6,752 B vs 6,752 B, yet 385 bytes differ in dynamic-symbol ordering.

BINARY EQUALITY

18 equal.
10 metadata differences.

A schematic diff of measured section status — not a raw byte dump.

.text ✓ equal	.plt ✓ equal	.rodata ✓ equal	.eh_frame ✓ equal	.eh_frame_hdr ✓ equal	.gcc_except_table ✓ equal	.init ✓ equal
.fini ✓ equal	.init_array ✓ equal	.fini_array ✓ equal	.got ✓ equal	.got.plt ✓ equal	.dynamic ✓ equal	.interp ✓ equal
.note.ABI-tag ✓ equal	.tm_clone_table ✓ equal	.comment ✓ equal	.shstrtab ✓ equal	.symtab △ differs	.strtab △ differs	.dynsym △ differs
.dynstr △ differs	.hash △ differs	.gnu.hash △ differs	.gnu.version △ differs	.gnu.version_r △ differs	.rela.dyn △ differs	.rela.plt △ differs

> .text is byte-identical at the same address in this measured pair.

Equal at the same addresses: .text, .plt, .rodata, .got.plt, and unwind sections. Differing dynamic metadata includes .dynsym and .rela.*; same 19 imports and PLT slots, different order.

REASONS FOR BINARY EQUALITY

Inlined after split.

main | greet | add | multiply | average | max_of<T>

```
int main() { add(...); multiply(...);
average(vals); greet(...); }
```

```
PLAIN-GC
plain-gc: one TU sees bodies → add/multiply/
average fold into main.

SPLIT-LTO-RELINK-GC
split: definitions owner holds main; ld.lld
-r ThinLTO sees all pieces → same folded
main.
```

Measured outcome: 178 normalized instructions and 3,314 B .text in each executable.

Summary: Identical executable code. Different symbol metadata. Not byte-identical executables.

Benchmarks ⚛

"Nuclear fission is the process in which the nucleus of a heavy atom splits into two or more smaller nuclei, releasing a massive amount of energy."

— Wikipedia, Nuclear fission

Benchmarks ⚛

"TU fission is the process in which the function bodies of a heavy TU splits into two or more smaller TUs, releasing a massive amount of TUs."

— me, cpp-splitter

Benchmark

Plain vs Split on EngFlow RBE

Boost.Spirit Test Suite

> Boost Spirit is an object-oriented, recursive-descent parser and output generation library for C++.

BOOST.SPIRIT ON ENGFLOW RBE

Boost.Spirit: 277 test programs, header-only, template-heavy.

```
// libs/spirit/test/qi/char1.cpp
#include <boost/spirit/include/qi_char.hpp>
#include <boost/spirit/include/
qi_action.hpp>
#include "test.hpp"

int main()
{
    using spirit_test::test;
    using namespace boost::spirit::ascii;

    BOOST_TEST(test("x", char_));
    BOOST_TEST(test("x", char_('x')));
    BOOST_TEST(!test("x", char_('y')));
    BOOST_TEST(test("x", char_('a', 'z')));
    BOOST_TEST(!test("x", char_('0',
'9')));
    BOOST_TEST(test(" ", ~char_('x')));
    return boost::report_errors();
}
```

```
// what a Spirit grammar looks like
#include <boost/spirit/include/qi.hpp>
namespace qi = boost::spirit::qi;

bool parse_numbers(std::string const& s,
                   std::vector<double>&
out)
{
    auto first = s.begin();
    bool ok = qi::phrase_parse(
        first, s.end(),
        qi::double_ % ',',   // the grammar
        qi::space,           // the skipper
        out);
    return ok && first == s.end();
}
```

279 translation units, each pulling in some 1,500 headers. The grammars are template expressions instantiated in every

```
        unit that spells them; the splitter's pieces are what is left once the templates stay in the preamble.
```

BOOST.SPIRIT ON ENGFLOW RBE

The edit: one line in one body, in a header 268 units include.

```cpp
// boost/spirit/home/support/char_encoding/standard_wide.hpp
namespace boost { namespace spirit { namespace char_encoding {
    struct standard_wide
    {
        // ... isalnum, isalpha, isdigit, tolower, toupper ...

        static ::boost::uint32_t
        toucs4(wchar_t ch)
        {
            (void)4;  // benchmark probe   <- the edit
            return static_cast<make_unsigned<wchar_t>::type>(ch);
        }
    };
}}}
```

plain

268 translation units recompile: every one that includes the header.

split, one piece per includer

267 pieces recompile, one per unit that keeps it; the copies and their PCHs are untouched.

split, one shared piece

51 compiles: `toucs4` once, with the other functions of its header and of the three headers that include it.

`toucs4` is named by templates in `char_class.hpp` and `qi/char/char.hpp`, so no unit can declare it only; it is emitted by one unit.

BENCHMARK SET 1 / CLUSTER -J500

End-to-end wall time. Two scenarios.

plain splitter

277 Boost.Spirit programs · 279 translation units · AMD EPYC-Milan (32 cores, 122 GiB) · clang 13.0.0 · C++17 Release · EngFlow RBE over mTLS · cpp-splitter 8ab6e9f3, 16 Sep 2026



Each bar is value ÷ this set's 1096.6s maximum. Full: plain is 1,641 remote build actions; split is 6,669 remote actions — 279 parses and every piece compiled on workers.

SET 1 / WHAT CHANGED

268 units reach it. 51 compiles, shared by all.

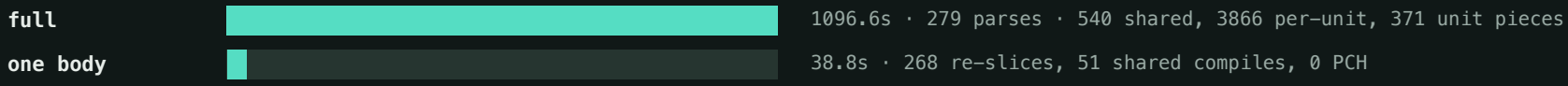
271	51	1575
PLAIN REMOTE COMPILES	SPLIT REMOTE COMPILES	SPLIT CACHED ACTIONS

The body-edit comparison is 136.2s versus 38.8s

Split 3.5x faster

BENCHMARK SET 2 / WHAT THE SPLITTER DID, -J500

What the splitter did.



Benchmark

Plain vs Split on EngFlow RBE

OpenCV core and imgproc

> OpenCV is the open-source computer vision library: matrices, image processing, filtering, geometry. Its `core` and `imgproc` modules are 158 translation units of ordinary C++ with the classes' inline members in `mat.inl.hpp`, a header all of them include.

OPENCV CORE AND IMGPROC

OpenCV: 158 units of ordinary C++, one inline header every unit includes.

```
// modules/core/include/opencv2/core/
mat.hpp
class CV_EXPORTS SparseMat
{
public:
    SparseMat();
    SparseMat(int dims, const int* sizes,
int type);
    //! returns the number of non-zero
elements
    size_t nzcount() const;
    //! computes the element hash value
    size_t hash(int i0) const;
    template<typename _Tp> _Tp& ref(int i0,
size_t* hashval = 0);
    // ...
    Hdr* hdr;
};

// modules/core/src/matrix_sparse.cpp
SparseMat::SparseMat(int _dims, const int*
_sizes, int _type)
    : flags(MAGIC_VAL), hdr(0)
{
    create(_dims, _sizes, _type);
}
```

```
// what a user writes
#include <opencv2/core.hpp>
#include <opencv2/imgproc.hpp>

cv::Mat edges_of(const cv::Mat& image)
{
    cv::Mat gray, blurred, edges;
    cv::cvtColor(image, gray,
cv::COLOR_BGR2GRAY);
    cv::GaussianBlur(gray, blurred,
cv::Size(5, 5), 1.4);
    cv::Canny(blurred, edges, 50, 150);
    return edges;
}
```

43 / 54

OpenCV 4.11.0, `core` and `imgproc`, static, `-DBUILD_LIST=core,imgproc`, nothing optional · 158 C++ units · clang 13, C++17, Release · every unit splits: 0 fallbacks, 0 declined



OPENCV CORE AND IMGPROC

The edit: one line in `SparseMat::nzcount()`, inline in a header all 158 units include.

```
// modules/core/include/opencv2/core/mat.inl.hpp
inline
size_t SparseMat::nzcount() const
{
    (void)4;  // benchmark probe   <- the edit
    return hdr ? hdr->nodeCount : 0;
}
```

plain

158 units recompile on the cluster, 139.1s: every one includes the header.

split, produced on the cluster

the same 158 re-slices and 23 pieces, 14.5s; the parse never runs here.

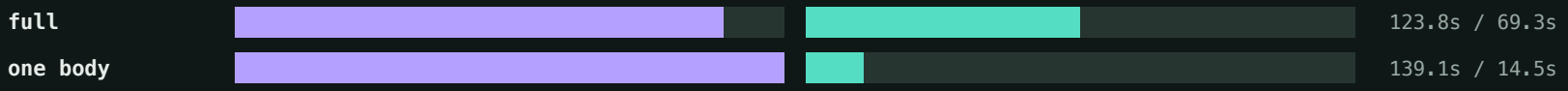
`nzcount` is called by `matrix_sparse.cpp` alone; the other 157 copies keep or declare it.

BENCHMARK SET 3 / OPENCV, CLUSTER -J500

OpenCV on the cluster: 9.6x on the edit.

plain splitter

158 C++ units · OpenCV 4.11.0 core and imgproc · AMD EPYC-Milan driving · clang 13.0.0 · C++17 Release · EngFlow RBE over mTLS, -j500 · split produced on the cluster



516 TUs plain / 26,436 split

SET 3 / WHAT CHANGED

158 units include it. 23 pieces compile.

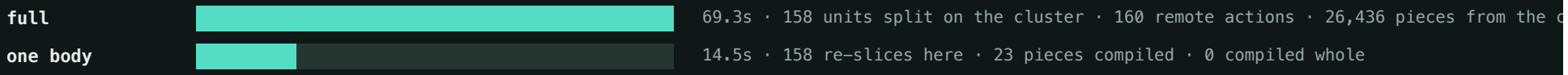
158	23	510
PLAIN REMOTE COMPILES	SPLIT REMOTE COMPILES	SPLIT CACHED ACTIONS

The body-edit comparison is 139.1s versus 14.5s

Split is 9.6x faster

SET 3 / WHAT THE SPLITTER DID, -J500

What the splitter did.



Every unit splits: static of an unnamed type, a static of a class in an unnamed namespace, a header included twice under two macro states -- and the edit re-slices all 158.

Benchmark

Unity vs Split Build on Local Machine

p4c - P4C_16 reference compiler

> Programming Protocol-independent Packet Processors (P4): domain-specific language for network devices, specifying how data plane devices (switches, NICs, routers, filters, etc.) process packets.

BENCHMARK SET 4 / P4C, HOST -J16, UNITY

p4c: 218 units, one header every unit includes

```
// lib/cstring.h: interned strings,
included by every unit
namespace P4 {
class cstring {
    const char *str = nullptr;
 public:
    // Size tests. Constant time except for
size(),
    // which is linear time.
    size_t size() const {
        return str ? strlen(str) : 0;
    }
    bool isNull() const { return str ==
nullptr; }
    const char *begin() const { return str;
}
    const char *end() const {
        return str ? str + strlen(str) :
str;
    }
```

```
    // Search for characters. Linear time.
    const char *find(int c) const {
        return str ? strchr(str, c) :
nullptr;
    }
    const char *findlast(int c) const {
        return str ? strrchr(str, c) : str;
    }
    cstring substr(size_t start, size_t
length) const;
    cstring trim(const char *ws = "
\t\r\n") const;
    // ... 60 more members
};
}  // namespace P4
```

p4c 8192431 without the control plane and the tests · C++20, libc++, Release · unity: `-DCMAKE_UNITY_BUILD=ON`, 29 batches of

8 · split: `cpp-splitter` as `CMAKE_CXX_COMPILER_LAUNCHER`, 0 fallbacks, 2 declined

49 / 54

P4C

The edit: one line in `cstring::findlast()`

```
// lib/cstring.h
    // Search for characters. Linear time.
    const char *find(int c) const { return str ? strchr(str, c) : nullptr; }
    const char *findlast(int c) const { (void)4; return str ? strrchr(str, c) : str; }
                                       ^^^^^^^^  the edit
```

plain

218 units recompile, 102.6s: every one includes the header.

unity

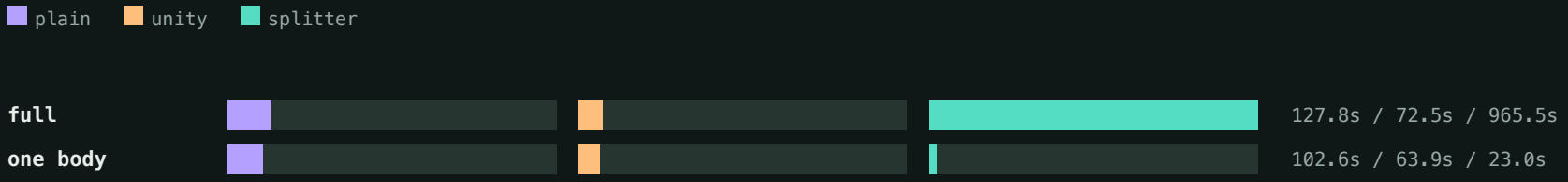
29 batches recompile, 63.9s: the header is in every batch.

split

2 units re-slice, 203 declare it only and change nothing: 2 PCHs, 50 pieces, 23.0s.

BENCHMARK SET 4 / P4C, HOST -J16

unity wins cold, the split wins every edit.



Build phase only, `-j16`, one run. One body is `cstring::findlast()`;

SET 4 / WHAT CHANGED

218 units impacted

50 pieces compile.

218	29	50
PLAIN UNITS RECOMPILED	UNITY BATCHES RECOMPILED	SPLIT PIECES COMPILED, 2 PCHS

The body edit is 102.6s plain, 63.9s unity, 23.0s split —

Split faster 4.5x over plain, 2.8x over unity.

IMPROVEMENTS

Next Steps

- There are no relocating linker on Windows
  - Achievable but requires implementing a special object file merger
- Adding lines in a function above others requires renumbering (and recompiling all functions below)
- Too much split pieces produced that are never used in complex builds
  - Uses a lot more disk space than necessary
- This is a very early prototype
- Bazel Integration: CMake Only for the moment

THANK YOU!

In a nutshell

github.com/daminetreg/splitter

⭐ ✨ Github stars welcome, alongside issues and PRs !💫

- **Very first** Cold split build is **much much more work**
  - **BUT** code-changes do not trigger full rebuild anymore, **Cache HIT rates increases drastically**
- **Iteration are much faster**: between **2.8x (p4c local)** to **9.6x (OpenCV on EngFlow RBE)**
  - AI Agents do alot of isolated edit to verify their changes
    - This accelerates AI based development in C++ drastically.
- Parallel caching compiler frontend for C++
  - Scales horizontally
  - Works with any existing compiler